

数据库事务（Transaction）系统教程

下面我不把它讲成一份“数据库理论课”，而是直接站在你现在的技术栈和项目上理解：

FastAPI + SQLAlchemy Async + PostgreSQL + Redis + AI Agent / MCP

你真正需要掌握的核心其实可以浓缩成一句话：

事务解决的是：一个业务操作涉及多次数据库修改时，如何让这些修改作为一个整体正确地完成。

而当进入并发场景后，还需要进一步解决：

多个请求同时修改同一份数据时，如何保证数据不会被错误地覆盖或超卖。

一、先理解：事务到底是什么？

假设你的舞蹈系统有一张会员卡：

会员卡
总次数：8
已使用：3
剩余：5

用户预约一节现代舞。

系统实际上不是只执行一条 SQL，而是可能执行：

① 创建预约记录

② 扣减会员卡次数

③ 减少课程剩余名额

④ 创建扣课记录

⑤ 更新相关业务数据

正常情况：

预约创建成功

↓

会员卡 -1

↓

课程人数 -1

↓

扣课记录创建成功

↓
COMMIT

全部成功。

但是如果：

- ① 创建预约成功
 - ② 扣课成功
 - ③ 课程人数更新成功
 - ④ 扣课记录创建失败

如果没有事务：

数据库：

预约：成功
会员卡：-1
课程名额：-1
扣课记录：失败

数据就不一致了。

用户可能发现：

“我的课时少了一节，但是预约记录有问题。”

二、事务解决的核心问题

事务就是把：

多个数据库操作

包装成：

一个不可分割的业务单元

例如：

BEGIN
 创建预约

```
扣会员卡次数  
更新课程名额  
创建扣课记录  
COMMIT
```

如果中间出现异常：

```
BEGIN  
  创建预约  
  扣会员卡次数  
  更新课程名额  
  创建扣课记录 ← 失败  
ROLLBACK
```

数据库恢复到事务开始之前的状态。

因此：

```
成功 → 全部生效  
  
失败 → 全部撤销
```

这就是事务最重要的价值。

三、ACID 到底是什么？

事务有四个经典特性：

A – Atomicity	原子性
C – Consistency	一致性
I – Isolation	隔离性
D – Durability	持久性

1. Atomicity：原子性

最简单的理解：

■ 要么全部成功，要么全部失败。

例如：

预约
↓
扣次数
↓
扣名额
↓
创建扣课记录

不能出现：

预约成功
扣次数成功
扣名额成功
扣课记录失败

事务会保证：

全部成功

或者：

全部回滚

原子性 $\neq$ 一致性

这是面试非常容易问的。

原子性：

这一组操作是不是整体执行？

一致性：

操作执行完之后，数据库是否仍然满足业务约束？

例如：

课程剩余名额 ≥ 0
会员卡剩余次数 ≥ 0
预约记录必须对应合法用户

事务主要提供原子性，但“一致性”还依赖：

- 数据库约束
- 外键
- 唯一约束
- CHECK
- 业务代码
- 并发控制

四、Consistency：一致性

假设你的系统规定：

```
remaining_count >= 0
```

那么下面这种情况就不应该发生：

```
remaining_count = -1
```

数据库可以通过：

```
CHECK (remaining_count >= 0)
```

防止非法数据。

再比如：

```
一个预约必须属于一个真实存在的用户
```

可以使用：

```
FOREIGN KEY
```

保证关系成立。

所以：

事务不是凭空创造一致性，而是帮助业务在一组操作完成后，从一个合法状态进入另一个合法状态。

五、Isolation：隔离性

隔离性是事务最难理解，也是后端开发非常重要的一部分。

假设：

会员卡剩余次数 = 1

同时来了两个请求：

用户 A → 预约
用户 B → 预约

两个事务：

Transaction A
Transaction B

它们同时操作数据库。

隔离性解决的问题就是：

一个事务执行过程中，另一个事务应该看到什么？

这就引出了：

- 脏读
- 不可重复读
- 幻读
- 丢失更新

后面会详细讲。

六、Durability：持久性

假设：

COMMIT

已经成功。

之后：

服务器突然断电
PostgreSQL 重启

数据仍然应该存在。

这就是持久性。

PostgreSQL 会通过 WAL（Write-Ahead Logging）等机制保障已经提交的数据能够在故障恢复后重新得到。

你不需要现在深入 PostgreSQL 存储引擎，但面试可以知道：

事务提交后，数据库通过持久化日志等机制保证已提交修改能够在故障恢复后恢复。

七、你的约课系统到底应该怎么使用事务？

假设：

用户
↓
预约现代舞
↓
会员卡扣 1 次
↓
课程剩余人数 - 1

生产环境可以设计成：

```
BEGIN
|
|— 查询课程
|
|— 检查课程是否可预约
|
|— 查询会员卡
|
|— 检查会员卡是否可使用
|
|— 创建预约
|
|— 扣减会员卡
|
|— 创建扣课记录
|
|— 更新课程预约人数
```

|
COMMIT

任何一步失败：

ROLLBACK

八、SQLAlchemy Async 怎么写？

你最应该掌握这个：

```
async with session.begin():  
    ...
```

例如：

```
async def book_class(  
    session: AsyncSession,  
    user_id: int,  
    class_id: int,  
    card_id: int,  
):  
    async with session.begin():  
        booking = Booking(  
            user_id=user_id,  
            class_id=class_id,  
        )  
        session.add(booking)  
  
        card = await get_membership_card(  
            session,  
            card_id,  
        )  
  
        card.remaining_count -= 1  
  
        lesson_record = LessonRecord(  
            user_id=user_id,  
            card_id=card.id,  
            booking_id=booking.id,  
            count=1,  
        )  
  
        session.add(lesson_record)
```

正常执行：

```
async with session.begin()  
    ↓  
BEGIN  
    ↓  
执行 SQL  
    ↓  
代码正常结束  
    ↓  
COMMIT
```

如果抛异常：

```
async with session.begin()  
    ↓  
执行 SQL  
    ↓  
发生 Exception  
    ↓  
ROLLBACK
```

这也是为什么生产代码里通常非常推荐：

```
async with session.begin():
```

而不是到处：

```
await session.commit()
```

九、commit() 到底做什么？

```
await session.commit()
```

核心作用：

提交当前事务，让修改正式生效。

例如：

```
card.remaining_count -= 1

await session.commit()
```

提交之后：

```
PostgreSQL
  ↓
事务提交
  ↓
其他事务可以看到提交后的结果
```

十、`rollback()` 是什么？

```
await session.rollback()
```

表示：

撤销当前事务中尚未提交的修改。

例如：

```
try:
    ...
    await session.commit()
except Exception:
    await session.rollback()
    raise
```

不过如果你已经使用：

```
async with session.begin():
```

通常不需要自己手动写 `rollback`：

```
async with session.begin():
    ...
```

异常退出时 SQLAlchemy 会负责回滚。

十一、`flush()` 和 `commit()` 的区别

这个非常重要。

假设：

```
booking = Booking(...)
session.add(booking)
```

此时对象可能还没有真正执行 INSERT。

调用：

```
await session.flush()
```

会把当前 Session 中待执行的 SQL 刷到数据库。

但：

flush ≠ commit

例如：

```
flush
↓
INSERT
↓
事务仍然没有提交
```

其他事务通常还不能把它当成已经提交的数据。

而：

```
commit
↓
事务正式提交
```

一个非常典型的场景

创建预约后需要拿到：

```
booking.id
```

可以：

```
session.add(booking)

await session.flush()

print(booking.id)
```

然后继续：

```
lesson_record = LessonRecord(
    booking_id=booking.id
)
```

最后：

```
await session.commit()
```

所以可以记：

flush = 把 SQL 发给数据库

commit = 提交事务

十二、Session 和 Transaction 到底是什么关系？

这是很多初学者最容易混淆的地方。

可以先这样理解：

```
FastAPI Request
  |
  ▼
AsyncSession
  |
  ▼
Transaction
  |
```

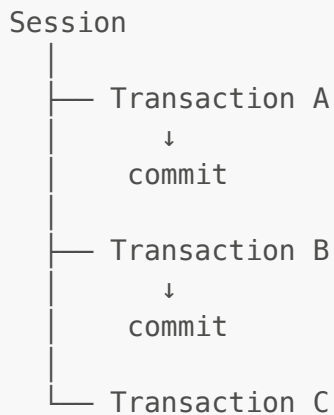
▼
PostgreSQL

但：

Session ≠ Transaction

`AsyncSession` 是 SQLAlchemy 的数据库会话/工作单元。

一个 Session 可以经历多个事务生命周期：



所以：

Session

是更高层的对象。

十三、一个请求应该对应一个 Session 吗？

通常推荐：

一个 Request
↓
一个 AsyncSession

例如：

```
async def get_db():
    async with async_session_factory() as session:
        yield session
```

FastAPI:

```
@router.post("/book")
async def book(
    session: AsyncSession = Depends(get_db),
):
    ...
```

这样:

```
HTTP Request
  ↓
AsyncSession
  ↓
Service
  ↓
Repository
  ↓
PostgreSQL
```

生命周期清晰。

十四、一个请求应该对应一个事务吗?

不一定。

这是一个很重要的区别。

如果这个 API 操作是一个完整业务单元:

预约课程

通常:

```
一个业务操作
  ↓
一个事务
```

非常合理。

但一个 HTTP 请求里面如果有多个完全独立的操作, 不一定必须放到一个事务里。

核心原则是:

事务边界应该围绕“业务原子性”设计，而不是机械地围绕 HTTP Request 设计。

十五、Repository 要不要 commit?

我比较推荐：

Repository 不负责 commit。

例如不推荐：

```
class BookingRepository:

    async def create(self, booking):
        self.session.add(booking)

        await self.session.commit()
```

因为这样 Service 很难组合多个 Repository。

例如：

```
BookingRepository
CardRepository
LessonRepository
```

如果每个 Repository 都：

```
commit
```

就变成：

```
创建预约
commit

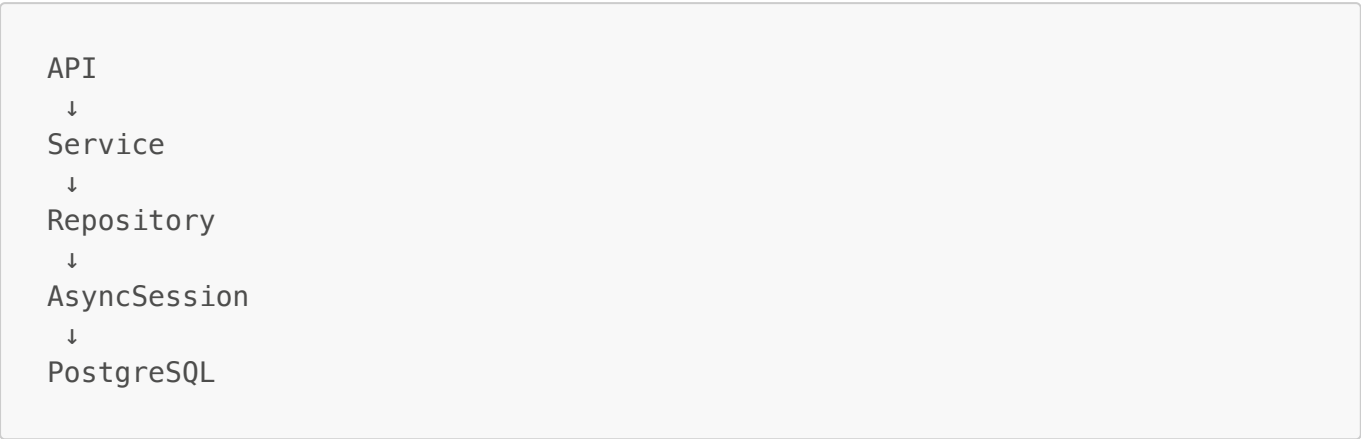
扣会员卡
commit

创建扣课记录
commit
```

这就破坏了原子性。

十六、更合理的架构

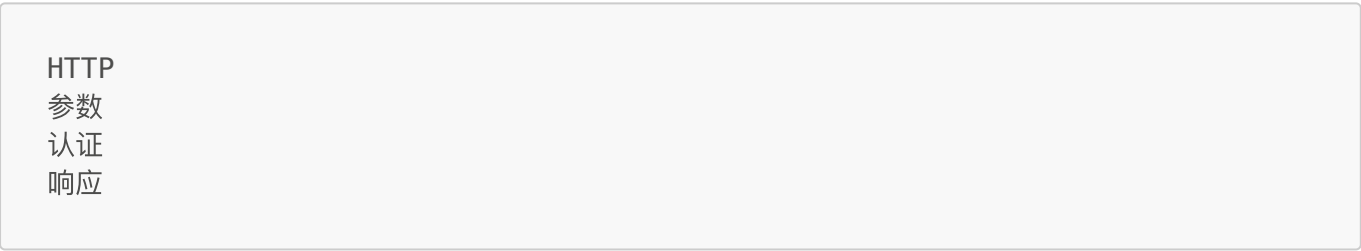
你的项目可以：



其中：

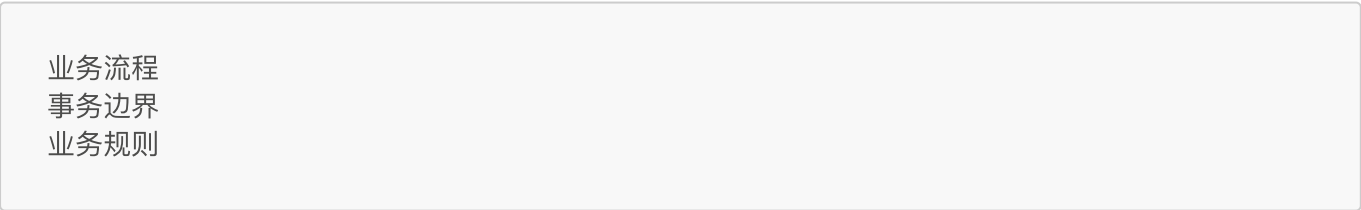
API

负责：



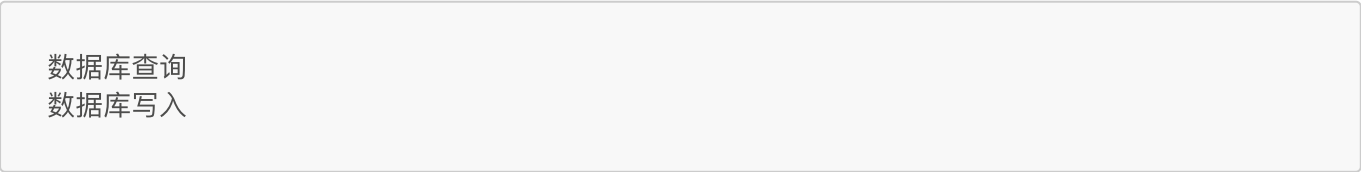
Service

负责：



Repository

负责：



例如：

```
async def book_class(...):
    async with session.begin():
        card = await card_repo.get_for_update(card_id)

        class_ = await class_repo.get_for_update(class_id)

        await booking_repo.create(...)

        await card_repo.consume(card)

        await lesson_repo.create(...)
```

这样：

Service 决定事务边界，Repository 不随意 commit。

这是你面试非常值得说的一点。

十七、PostgreSQL 为什么能支持并发？

这里要理解一个核心：

MVCC: Multi-Version Concurrency Control

多版本并发控制。

简单理解：

数据库并不是简单地：

只有一份数据

而是通过版本信息让不同事务在一定程度上看到属于自己的数据版本。

例如：

```
Transaction A
读取数据

Transaction B
修改数据
```

A 不一定需要完全阻塞 B。

这也是 PostgreSQL 高并发能力的重要基础。

你现在不需要深入 PostgreSQL 源码，但一定要理解：

```
PostgreSQL
  ↓
MVCC
  ↓
并发事务
  ↓
减少读写之间不必要的阻塞
```

十八、事务隔离级别

SQL 标准定义了：

```
Read Uncommitted
Read Committed
Repeatable Read
Serializable
```

但 PostgreSQL 有一个重要特点：

PostgreSQL 实际上不会提供真正意义上的 Read Uncommitted；它会按 Read Committed 的行为处理。

你面试时可以直接记住。

十九、Read Committed

PostgreSQL 默认隔离级别：

```
READ COMMITTED
```

核心理解：

一个 SQL 查询只能看到其他事务已经提交的数据。

例如：

```
Transaction A
UPDATE card SET remaining = 5
```

但是还没有：

```
COMMIT
```

此时：

```
Transaction B  
SELECT remaining
```

不会看到 A 尚未提交的修改。

所以避免：

脏读。

二十、什么是脏读？

例如：

```
A: remaining = 5  
B: 修改为 4  
B: 还没有 commit  
  
A: 读取到 4
```

如果 B：

```
ROLLBACK
```

那么 A 刚才读到的：

```
4
```

其实根本没有真正存在。

这就是：

脏读。

PostgreSQL 的 Read Committed 不允许这种情况。

二十一、不可重复读

例如：

```
Transaction A  
第一次查询：  
remaining = 5
```

然后：

```
Transaction B  
修改：  
remaining = 4  
COMMIT
```

A 再查询：

```
remaining = 4
```

同一个事务里：

```
第一次：5  
第二次：4
```

这就是：

不可重复读。

二十二、Repeatable Read

Repeatable Read 可以让事务在一个更稳定的数据快照上工作。

简单理解：

```
Transaction A  
  ↓  
建立快照  
  ↓  
查询 remaining = 5  
  ↓  
Transaction B 修改
```

↓
A 再查询
↓
仍然基于自己的事务快照

不过 PostgreSQL 的 Repeatable Read 行为比简单的 SQL 标准描述更强，它基于事务快照，并可能在并发更新冲突时产生 serialization failure，需要应用重试。

二十三、Serializable

最高级别：

SERIALIZABLE

目标是：

让并发事务的最终效果等价于某种串行执行。

例如：

A
B

并发执行。

数据库发现：

这种并发结果可能破坏串行化

可能让其中一个事务失败。

应用需要：

捕获异常
↓
重新执行事务

代价就是：

并发冲突下性能和重试成本更高。

二十四、真正重要的问题：两个用户抢最后一个名额

假设：

```
课程剩余名额 = 1
```

同时：

```
用户 A → 预约  
用户 B → 预约
```

如果代码是：

```
class_.remaining -= 1
```

可能发生：

```
A: 读取 1  
B: 读取 1  
  
A: 计算 0  
B: 计算 0  
  
A: 写 0  
B: 写 0
```

最终：

```
remaining = 0
```

看起来没问题。

但：

```
A: 预约成功  
B: 预约成功
```

实际上卖出了两份。

这就是并发业务问题。

二十五、事务本身不一定能解决并发超卖

这是非常重要的一点：

“加了事务” ≠ “并发安全”。

事务可以保证：

A 的操作是原子的
B 的操作是原子的

但：

A 和 B 同时竞争同一资源

还需要并发控制。

二十六、最直接的方法：FOR UPDATE

例如：

```
SELECT *  
FROM membership_cards  
WHERE id = $1  
FOR UPDATE;
```

含义：

查询这行数据，并对它加行级排他锁。

假设：

```
Transaction A  
SELECT ... FOR UPDATE
```

获得锁。

此时：

```
Transaction B
SELECT ... FOR UPDATE
```

需要等待。

于是：

```
A
↓
读取次数 1
↓
扣成 0
↓
COMMIT
↓
释放锁

B
↓
拿到锁
↓
读取最新数据
↓
发现剩余 0
↓
失败
```

这才是我们想要的结果。

二十七、SQLAlchemy Async + FOR UPDATE

例如：

```
from sqlalchemy import select

result = await session.execute(
    select(MembershipCard)
    .where(MembershipCard.id == card_id)
    .with_for_update()
)

card = result.scalar_one()
```

然后：

```
if card.remaining_count <= 0:
    raise BusinessError("课时不足")

card.remaining_count -= 1
```

整个过程必须在事务中：

```
async with session.begin():
    result = await session.execute(
        select(MembershipCard)
        .where(MembershipCard.id == card_id)
        .with_for_update()
    )

    card = result.scalar_one()

    if card.remaining_count <= 0:
        raise BusinessError("课时不足")

    card.remaining_count -= 1
```

这就是非常典型的：

事务 + 行锁

二十八、条件 UPDATE：很多时候更简单

还有一种非常实用的方法：

```
UPDATE membership_cards
SET remaining_count = remaining_count - 1
WHERE id = :card_id
AND remaining_count > 0;
```

然后检查：

```
affected_rows == 1
```

成功：

扣课成功

失败：

没有剩余次数

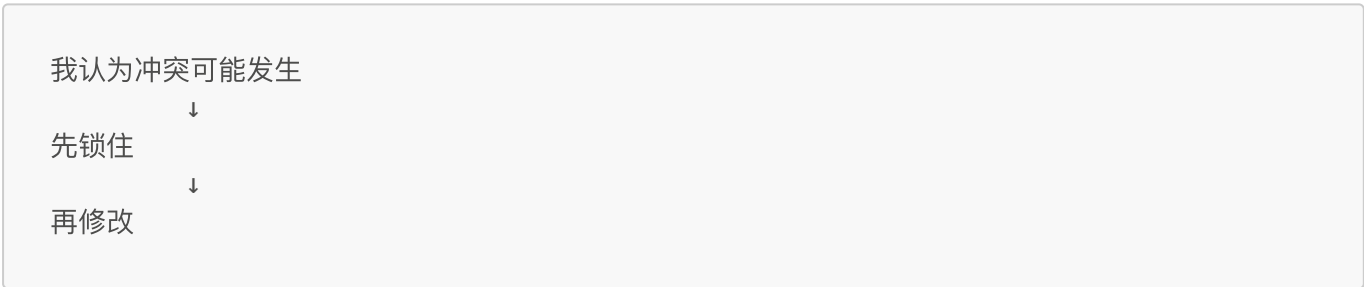
这种方式非常适合简单的库存/次数扣减。

它的优势是：

把“检查 + 扣减”变成一个原子 SQL 操作。

二十九、悲观锁 vs 乐观锁

悲观锁



典型：

FOR
UPDATE

适合：

抢名额
扣库存
扣课时
资金余额

乐观锁

增加：

```
version
```

例如：

```
version = 5
```

更新：

```
UPDATE membership_cards
SET remaining_count = 4,
    version          = 6
WHERE id = 1
AND version = 5;
```

如果：

```
affected_rows = 1
```

成功。

如果：

```
affected_rows = 0
```

说明数据已经被其他请求修改。

三十、Redis 分布式锁是不是更好？

不是。

这是一个非常常见的误区：

看到并发问题就上 Redis Lock。

你的核心业务数据：

```
课程名额
会员卡次数
```

预约记录
扣课记录

本身就在：

PostgreSQL

那么优先应该利用：

PostgreSQL Transaction
+
Row Lock
+
Unique Constraint
+
Conditional UPDATE

解决。

Redis 更适合：

分布式锁
限流
缓存
Token
短期状态
分布式协调

而不是替代 PostgreSQL 的事务。

三十一、Redis 和 PostgreSQL 事务是什么关系？

这是 AI Agent 后端非常重要的问题。

假设：

PostgreSQL COMMIT 成功

但是：

Redis SET 失败

那么：

PostgreSQL：成功
Redis：失败

PostgreSQL 的事务无法自动：

ROLLBACK Redis

因为：

PostgreSQL

和：

Redis

是两个独立系统。

三十二、不要把外部系统放进核心数据库事务里

例如不推荐：

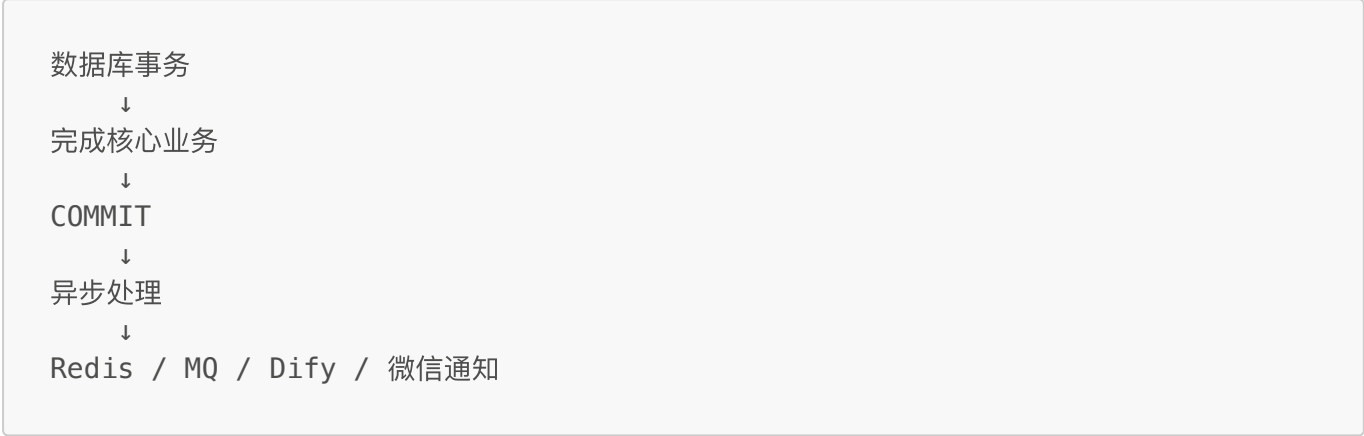
```
async with session.begin():  
    await create_booking()  
  
    await card.consume()  
  
    await call_dify()  
  
    await call_wechat_api()  
  
    await send_email()
```

因为外部 API：

可能慢
可能超时
可能失败

导致数据库事务长时间持有锁。

更合理：

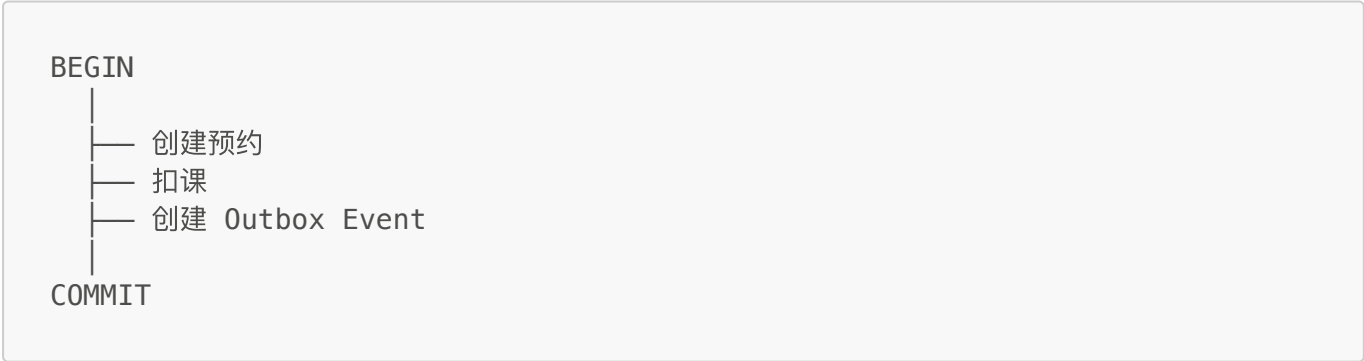


三十三、Outbox Pattern

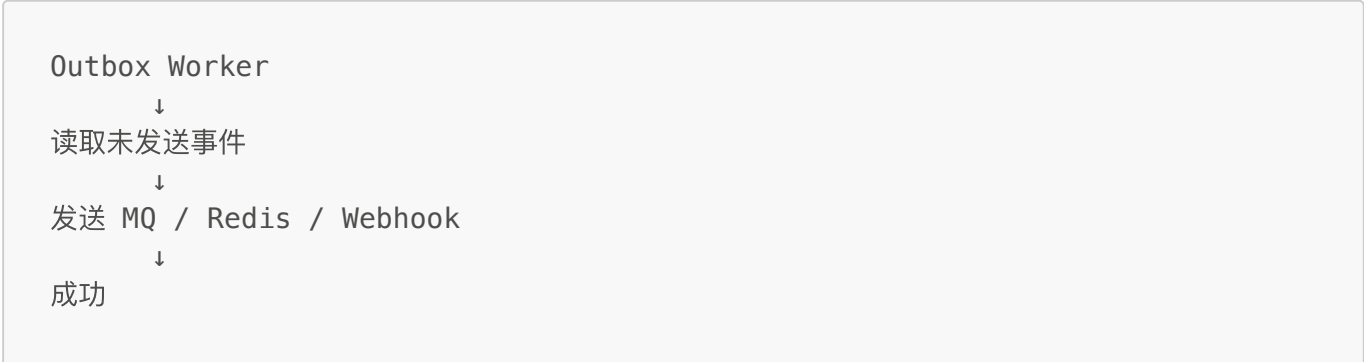
如果你需要保证：

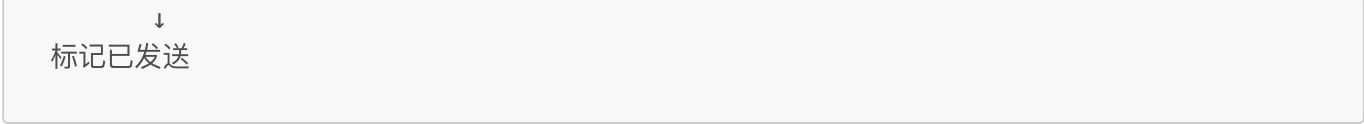
数据库业务成功后，一定最终产生消息。

可以使用：

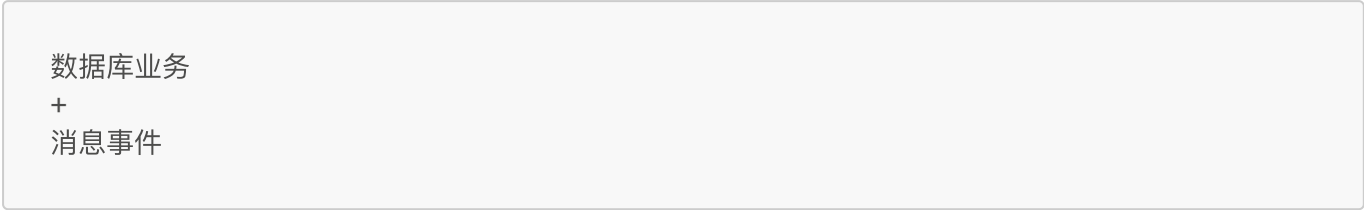


然后：



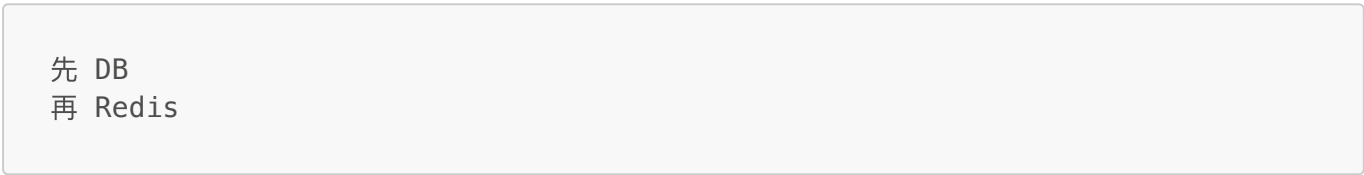


这样：



在同一个 PostgreSQL 事务中提交。

这比：



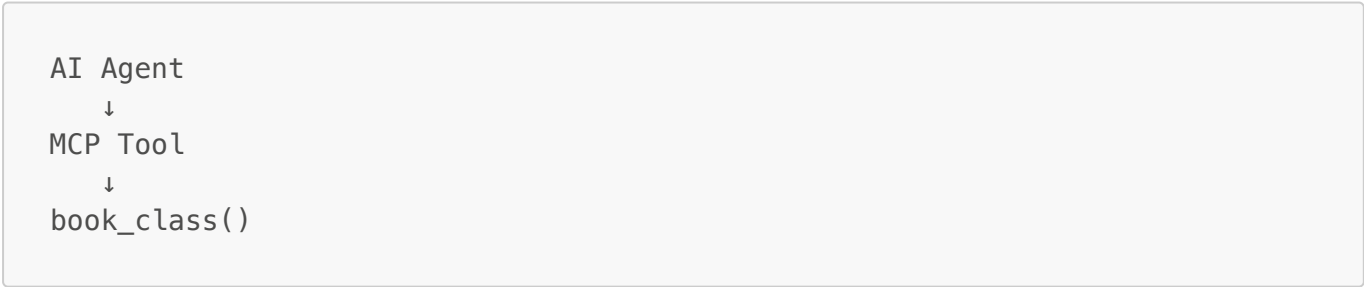
可靠得多。

三十四、AI Agent / MCP 为什么特别需要事务？

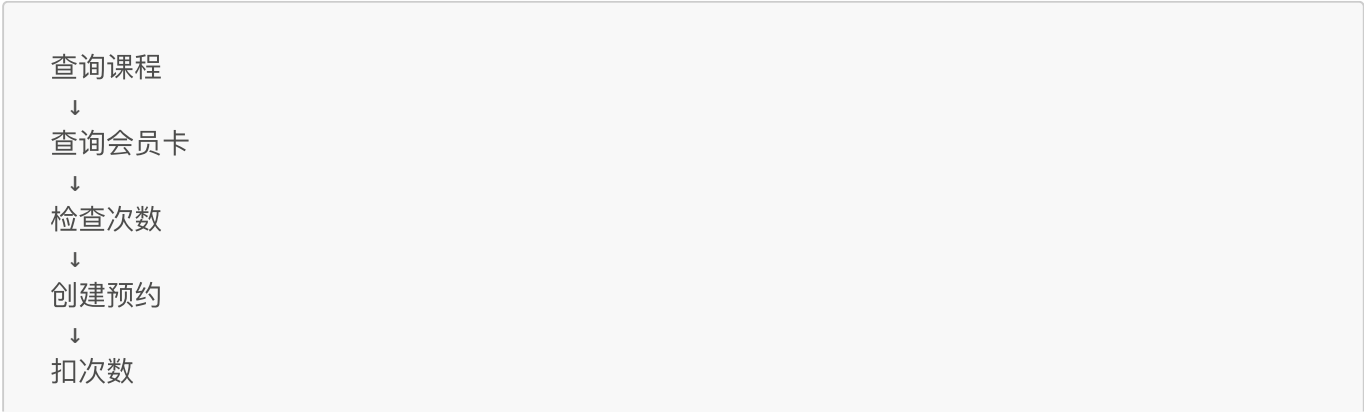
假设用户说：

帮我预约明天晚上 7 点的现代舞。

AI Agent：



Tool 内部：



↓
创建扣课记录

这些属于：

一个业务原子操作

因此：

book_class()

内部应该拥有自己的事务边界。

三十五、Agent 重复调用怎么办？

这是 AI Agent 和普通 Web API 一个非常重要的区别。

例如：

Agent
↓
book_class()

请求成功了。

但是网络超时：

Agent 没收到响应

于是 Agent 再次调用：

book_class()

如果没有幂等设计：

预约一次
扣一次

再次预约
再扣一次

用户直接被扣两次。

三十六、事务和幂等不是一回事

这个一定要分清：

事务解决

一次调用内部：

A
B
C

要么全部成功
要么全部失败

幂等解决

同一个业务请求执行一次
执行多次

最终结果仍然应该一样

所以：

事务
+
幂等

是两个不同维度。

三十七、如何给预约 API 做幂等？

可以设计：

`idempotency_key`

例如 Agent：

```
booking_request_id  
=  
agent_20260919_xxx
```

数据库：

```
UNIQUE(user_id, idempotency_key)
```

第一次：

创建成功

第二次：

唯一约束冲突

或者查询已有预约：

```
已经处理过  
↓  
直接返回原来的结果
```

三十八、数据库唯一约束非常重要

例如：

```
同一个用户  
同一个课程  
同一个时间
```

不允许产生重复预约。

可以根据你的业务设计：

```
UNIQUE(user_id, class_schedule_id)
```

但这里要注意：

唯一约束必须严格根据真实业务规则设计。

例如允许用户取消后重新预约，就不能简单地永久禁止相同组合。

这时候可能需要：

状态字段

或者 PostgreSQL 的：

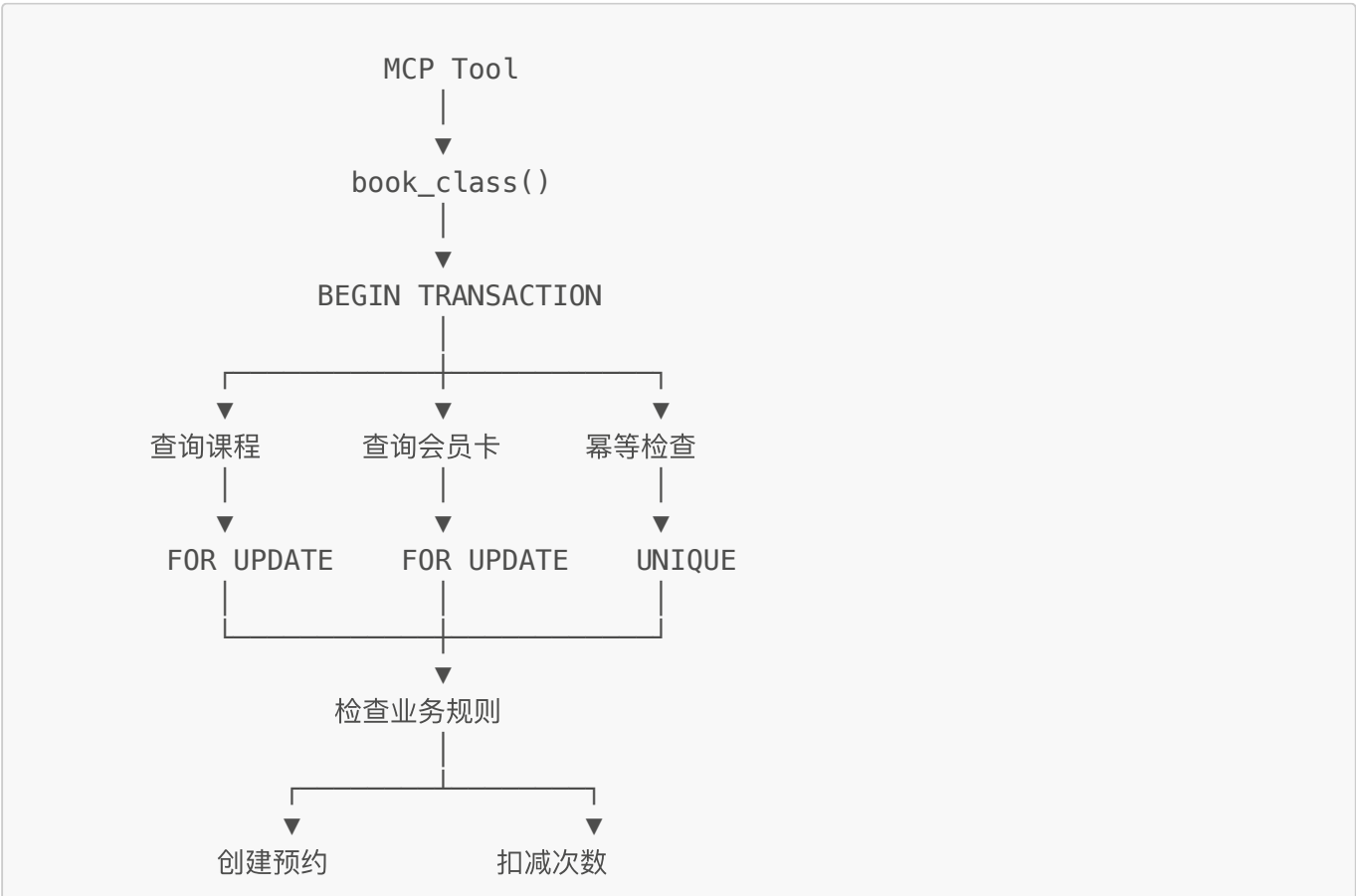
Partial Unique Index

例如只约束：

`status = 'booked'`

的记录。

三十九、你的约课事务可以最终设计成这样



```
graph TD; A[ ] --> B[创建扣课记录]; B --> C[COMMIT]; C --> D[返回预约结果];
```

这已经是一个非常典型的生产级业务事务。

四十、20% 核心知识：你现在最应该掌握什么？

如果把整个事务压缩到最重要的 20%，你先掌握下面这些：

第一层：必须懂

```
Transaction
ACID
BEGIN
COMMIT
ROLLBACK
```

第二层：必须会写

```
async with session.begin():
    ...
```

第三层：必须理解

```
Session ≠ Transaction
flush ≠ commit
```

第四层：后端必须掌握

```
并发
锁
FOR UPDATE
```

条件 UPDATE
唯一约束

第五层：生产环境必须掌握

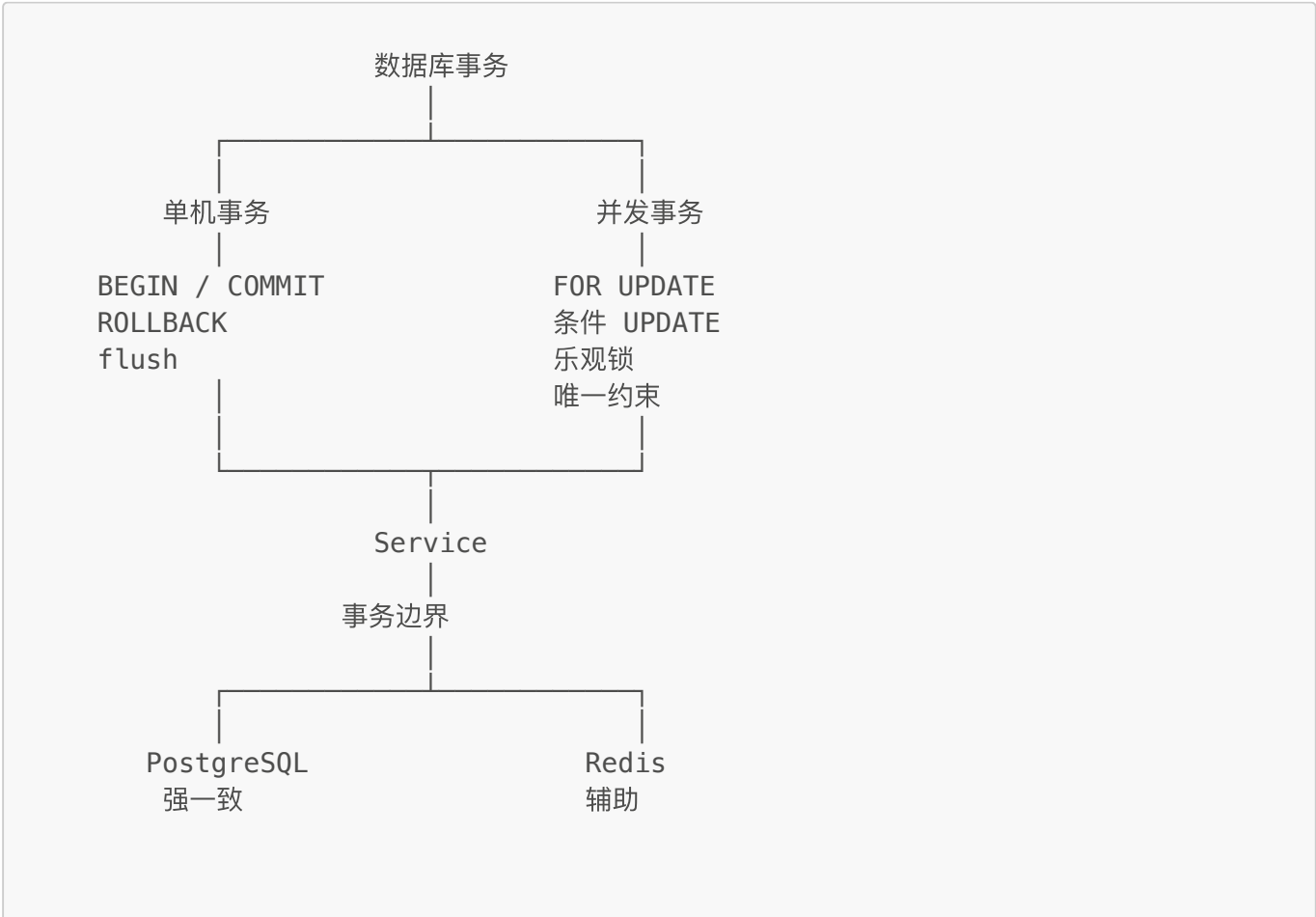
事务边界
Service 管事务
Repository 不随意 commit

第六层：AI Agent 必须增加

幂等
重试
唯一约束
Outbox
最终一致性

四十一、80% 实际后端开发最常用的知识

对于你目前的 FastAPI 项目，我认为最值得投入时间的是：





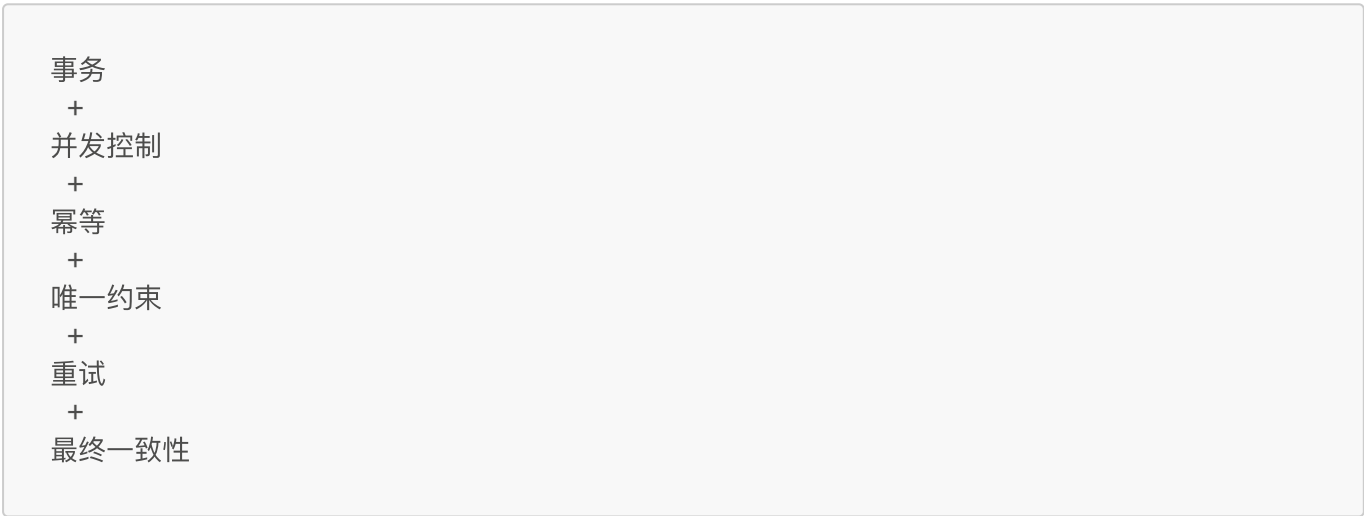
这部分基本就是你现在最应该掌握的。

四十二、AI Agent / MCP 必须额外掌握

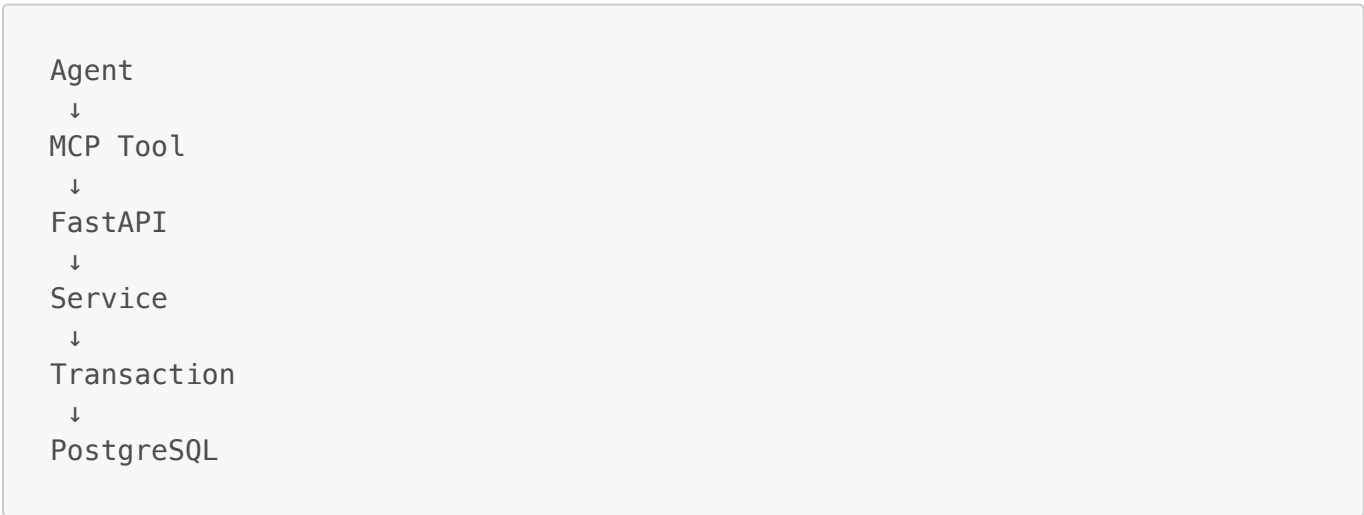
AI Agent 场景比传统 CRUD 多了一个问题：

Agent 可能重复调用工具。

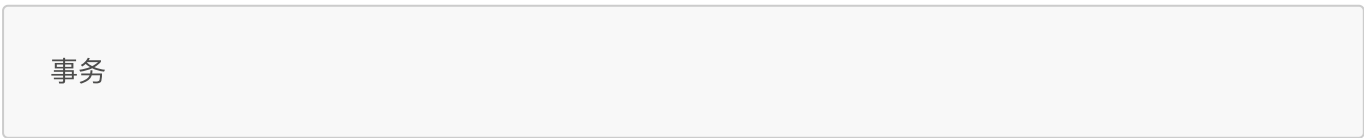
所以你的知识体系应该变成：



例如：



其中：



保证一次业务操作内部的一致性。

幂等

保证 Agent 重试不会产生重复业务结果。

唯一约束

让数据库成为最终的数据安全边界。

四十三、面试：30 秒回答“什么是数据库事务？”

可以直接这样回答：

数据库事务是一组不可分割的数据库操作单元，通过 ACID 保证这组操作能够可靠地完成。比如我的约课系统中，一次预约可能同时涉及创建预约、扣减会员卡次数、更新课程名额和创建扣课记录，这些操作需要放在同一个事务中，要么全部提交，要么出现异常时全部回滚。对于并发场景，我还会结合 PostgreSQL 的行锁、条件更新和唯一约束解决资源竞争问题。

这已经是比较完整的后端面试回答。

四十四、面试：1 分钟回答“你项目中怎么使用事务？”

可以这样说：

我的项目使用 FastAPI + SQLAlchemy Async + PostgreSQL。事务主要在 Service 层围绕业务操作进行管理，Repository 层主要负责数据访问，不在 Repository 中随意 commit。比如用户预约课程时，我会在一个事务中完成课程检查、会员卡检查、创建预约、扣减课时以及创建扣课记录。如果中间任何一步发生异常，事务自动 rollback，避免出现预约创建成功但扣课失败这种数据不一致。对于会员卡扣次和课程名额这种并发敏感的数据，我会结合 SELECT FOR UPDATE 或条件 UPDATE，以及数据库唯一约束进行并发控制。

四十五、面试：2 分钟回答“两个用户抢最后一个课程名额怎么办？”

可以这样回答：

首先我不会认为简单增加事务就可以解决这个问题，因为事务解决的是一组操作的原子性，并不自动解决多个事务之间的资源竞争。

**例如课程只剩一个名额，两个请求同时读取到 remaining=1，如果只是普通查询然后执行减一，就可能出现两个请求都预约成功的问题。 **

我的处理方式之一是在事务中对课程记录使用 PostgreSQL 的 `SELECT FOR UPDATE` 获取行锁。第一个事务获得锁后检查剩余名额并扣减，第二个事务需要等待第一个事务提交，之后重新读取最新数据，如果发现剩余名额为 0，就返回课程已满。对于简单的扣库存场景，也可以使用 `UPDATE ... SET remaining = remaining - 1 WHERE remaining > 0`，然后根据 `affected rows` 判断是否扣减成功。

同时我会增加数据库唯一约束防止同一个用户产生重复预约。对于 AI Agent / MCP 场景，还需要增加幂等键，因为 Agent 可能由于网络超时重复调用预约工具。

这个回答已经覆盖：

- 事务
- 并发
- 锁
- 条件 UPDATE
- 唯一约束
- 幂等
- AI Agent

非常适合你现在的目标岗位。

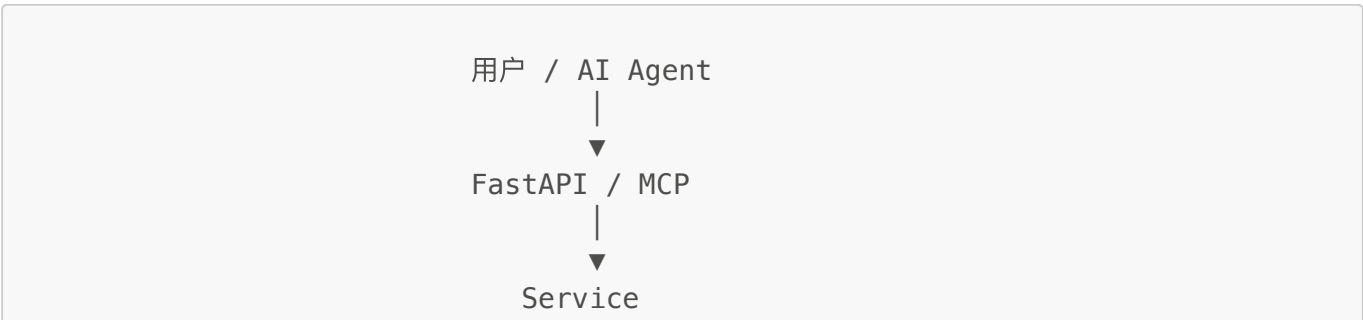
四十六、面试：2 分钟回答“Redis 和数据库事务怎么保证一致性？”

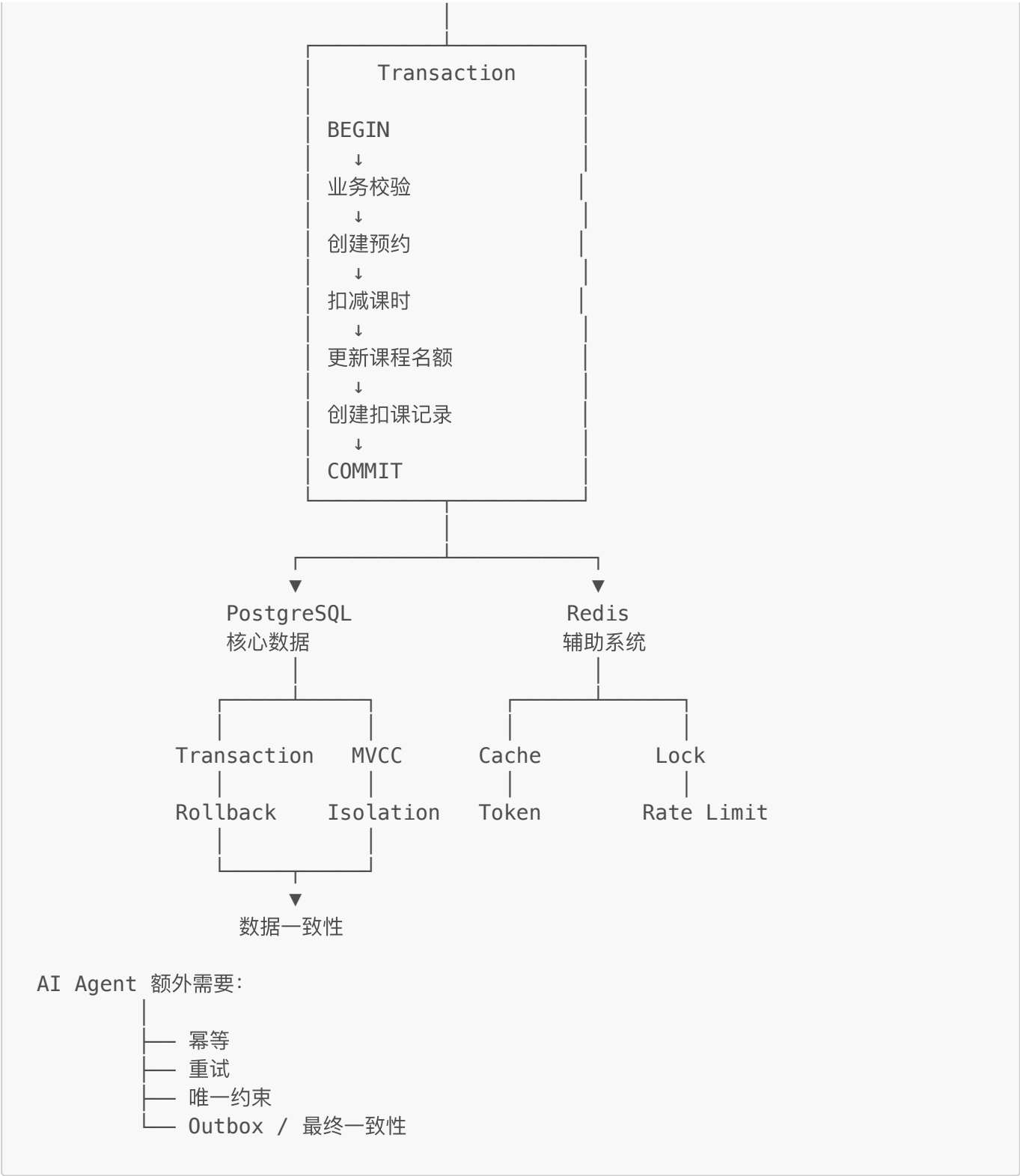
可以回答：

Redis 和 PostgreSQL 属于两个独立的数据系统，PostgreSQL 的事务不能把 Redis 操作自动纳入同一个 ACID 事务，所以我不会把 Redis 和 PostgreSQL 简单理解成一个事务。核心业务数据，例如预约、会员卡次数、扣课记录，我会优先放在 PostgreSQL 中，通过数据库事务保证一致性。Redis 更多承担缓存、限流、Token、分布式协调等职责。

****如果确实需要在数据库提交后产生 Redis 或消息事件，我会考虑 Outbox Pattern。也就是在 PostgreSQL 事务中同时写入业务数据和 Outbox Event，事务提交成功后，由后台 Worker 异步读取 Outbox 并发送消息。这样即使 Redis 或消息系统临时失败，也可以通过重试实现最终一致性。****

四十七、最后给你一张真正值得记住的图





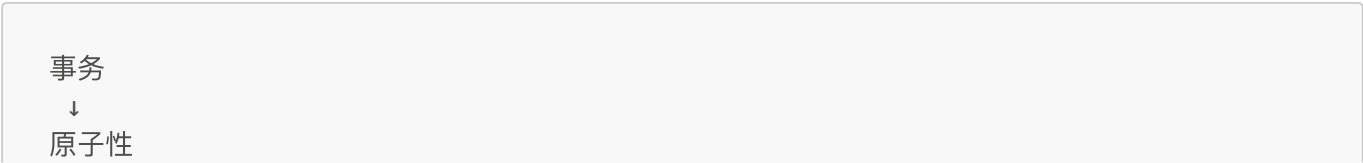
你现在真正需要建立的认知

不要把数据库事务理解成：

```
session.commit()
```

 的使用方法。

对于你从前端转 Python 后端的学习路线，更应该建立下面这条完整链路：



↓
事务边界
↓
并发
↓
锁
↓
数据库约束
↓
幂等
↓
分布式一致性
↓
AI Agent Tool 重试

其中你当前项目最值得真正写进代码、跑测试、制造并发场景验证的，就是：

“两个用户同时预约最后一个名额 + 两个请求同时扣最后一次课时 + Agent 重复调用预约 Tool”

把这三个场景真正做出来，你对事务的理解会从“知道概念”进入生产后端工程实践。